CAR PARKING SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

KEERTHANA R – 810023205068

.

DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Car Parking System"** is the Bonafide work of **KEERTHANA R (810023205068)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The **Car Parking Management System** is a full-stack web application developed to modernize and simplify the management of parking facilities. The system provides a centralized digital platform where users can register and log in, check available parking spaces, reserve parking slots, and manage their parking details. It also enables administrators to monitor parking spaces, manage user bookings, track vehicle information, and oversee parking-related activities efficiently.

The project is implemented using **Java with Spring Boot** for the backend, providing a scalable and reliable architecture for handling business logic and RESTful web services. The frontend is developed using **HTML, CSS, JavaScript, and Bootstrap**, providing a responsive and user-friendly interface. The integration of these technologies enables smooth communication between the frontend and backend while ensuring efficient management of parking operations.

One of the main objectives of the system is to **reduce manual parking management, minimize booking errors, avoid unnecessary waiting time, and improve the utilization of available parking spaces**. By digitizing processes such as slot allocation, reservation, vehicle management, and parking monitoring, the system improves operational efficiency and provides a convenient experience for users.

Overall, the **Car Parking Management System** demonstrates the practical application of full-stack development and modern software engineering principles. It combines a Java-based backend with an interactive web interface to provide an effective solution for managing parking facilities. The project

highlights how technology can be used to automate parking operations, improve resource utilization, and enhance the overall efficiency of parking management.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP	1 - 8
2	COMPANY PROFILE	8 - 11
3	DAILY WORK REPORT	12 - 15
4	BLOCK DIAGRAM	16
5	TRAVEL AGENCY MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILIITIES	16 - 19
6	PROPOSED METHOD 6.1 USER AUTHENTICATION 6.2 PACKAGE MANAGEMENT 6.3 VEHICLE MANAGEMENT 6.4 PAYMENT PROCESSING 6.5 ADMIN MANAGEMENT	20 - 21

7	ALGORITHM 7.1 USER LOGIN 7.2 USER REGISTRATION 7.3 PARKING SLOT BOOKING 7.4 VEHICLE MANAGEMENT 7.5 PAYMENT PROCESSING 7.6 BOOKING MANAGEMENT 7.7 ADMIN OPERATIONS	21 - 23
8	CODE IMPLEMENTATION	23 - 29
9	SAMPLE OUTPUT	30 - 33
10	ANALYSIS	34
11	CONCLUSION	35
12	FUTURE WORK	35 - 36
13	REFERENCES	36

1.INTRODUCTION

The Car Parking Management System is a Java Full Stack web application developed to automate and simplify the day-to-day operations of parking facilities. In today's rapidly growing urban environment, the increasing number of vehicles has

created a significant demand for efficient and organized parking management. Traditional parking systems that depend on manual record keeping, paper-based tickets, and physical monitoring often face problems such as long waiting times, parking slot mismanagement, inaccurate records, booking conflicts, and difficulty in tracking vehicle information. To overcome these challenges, a centralized and webbased **Car Parking Management System** has been developed.

The primary purpose of this project is to provide a secure and user-friendly platform where users can register and log in, check the availability of parking slots, reserve their preferred parking space, provide vehicle details, and manage their parking bookings. The system also provides an administrator panel that enables administrators to manage users, parking slots, vehicle information, bookings, payments, and parking-related activities efficiently. This automation reduces manual work, improves data accuracy, minimizes parking conflicts, and enhances the overall user experience.

The application is developed using **Java Full Stack technologies**. The frontend is designed using **HTML5, CSS3, JavaScript, and Bootstrap 5**, which provide an attractive, responsive, and interactive user interface. The backend is developed using **Java Spring Boot**, which handles business logic, REST API development, user authentication, parking slot management, booking operations, and communication between the frontend and database. **MySQL** is used as the relational database management system to store user details, vehicle information, parking slot details, booking records, payment information, and other parking-related data. **Apache Maven** is used for dependency management and project building, while **IntelliJ IDEA/Eclipse** can be used as the development environment.

The system follows a **three-tier architecture**, consisting of the Presentation Layer

(Frontend), Business Logic Layer (Spring Boot Backend), and Database Layer (MySQL). The frontend communicates with the backend through RESTful APIs, allowing efficient data exchange and smooth interaction between users and the application. This layered architecture improves the maintainability, scalability, reliability, and security of the system.

The **Car Parking Management System** consists of several important modules, including **User Registration and Login, Parking Slot Management, Vehicle Management, Parking Booking Management, Payment Management, and Admin Management**. Each module performs specific functions and works together to provide a complete parking management solution. The system can also implement role-based access, allowing users and administrators to access only the features relevant to their roles.

Overall, the **Car Parking Management System** is a practical implementation of Java Full Stack Development that demonstrates the integration of frontend technologies, Spring Boot backend services, and MySQL database management. The project provides practical knowledge in web application development, database connectivity, REST API implementation, authentication, and software engineering principles. It serves as an efficient, scalable, and user-friendly solution for modern parking facilities and can be further enhanced with features such as **real-time parking availability, QR-based entry and exit, automatic number plate recognition, online payment gateways, GPS-based parking location, and mobile application support** in future versions.

1.1 OBJECTIVES OF THE INTERNSHIP

Specific Objectives

The main objective of the **Car Parking Management System** is to develop a secure, user-friendly, and efficient web application using **Java Full Stack** technologies. The system is designed to simplify parking facility operations by providing an online platform for users and administrators to manage parking spaces, vehicle information, and parking bookings effectively.

The objectives of the project are:

- To develop a web-based application using **Java Spring Boot, HTML, CSS, JavaScript, Bootstrap, and MySQL**.
- To provide secure **user registration and login** functionality.
- To enable users to **view available parking slots** in real time.
- To allow users to **reserve and manage parking slots** conveniently.
- To maintain and manage **vehicle and user information** securely.
- To help administrators manage **users, parking slots, vehicles, and bookings**.
- To maintain accurate records of **parking reservations and payment details**.
- To reduce manual parking management and minimize **booking errors and slot conflicts**.
- To improve the efficient utilization of **available parking spaces**.

- To provide a **responsive, reliable, and easy-to-use** parking management application.
- To enhance user convenience by reducing **waiting time and manual procedures**.
- To demonstrate the practical application of **Java Full Stack development, REST APIs, database connectivity, and software engineering principles**.

1.2 MOTIVATION

The motivation behind developing the **Car Parking Management System** is to make parking operations faster, easier, and more efficient. Many parking facilities still depend on manual methods to manage parking slots, vehicle details, and bookings, which can lead to delays, inaccurate records, parking conflicts, and unnecessary waiting time.

This project provides an online platform where users can easily **check available parking slots, reserve parking spaces, manage vehicle details, and view their booking information**. It also helps administrators manage parking slots, users, vehicles, bookings, and other parking-related activities from a single centralized system.

Another major motivation is to **reduce paperwork, minimize manual errors, improve parking space utilization, and provide better convenience to users**. By automating important parking operations, the system helps save time and improves the overall efficiency of parking management.

This project also provides practical knowledge of **Java Full Stack Development** by using technologies such as **Spring Boot, HTML, CSS, JavaScript, Bootstrap, and MySQL** to develop a real-world web application. It helps demonstrate how modern software technologies can be applied to solve common parking management problems effectively.

2.COMPANY PROFILE



About the Company:

Fantasy Solution is an Information Technology (IT) company that focuses on providing software development and digital technology solutions to meet the growing needs of modern businesses. The company specializes in developing web applications, software solutions, and technology-based services that help organizations improve their operational efficiency and adopt digital solutions. The primary objective of Fantasy Solution is to develop reliable, user-friendly, and efficient software applications that address real-world business requirements. The company follows modern software development practices and focuses on

application design, development, database management, testing, and implementation. It also emphasizes quality, performance, maintainability, and customer satisfaction throughout the development process.

Fantasy Solution also provides industrial training and internship opportunities for students and fresh graduates. These programs help students bridge the gap between academic knowledge and practical industry requirements by providing exposure to real-time software development projects. During the internship, students can gain practical experience in areas such as frontend development, backend development, database management, REST API development, software testing, debugging, and application deployment.

As part of my internship at Fantasy Solution, I developed a Car Parking Management System using Java Full Stack technologies. The project involved designing a responsive and user-friendly interface using HTML, CSS, JavaScript, and Bootstrap, developing backend services using Java Spring Boot, integrating the MySQL database, implementing RESTful APIs, and testing the application. The system was designed to simplify parking management by providing features such as user registration and login, parking slot management, vehicle management, parking slot booking, and administrative management.

The internship provided me with valuable practical knowledge of Java Full Stack Development and helped me understand the complete Software Development Life Cycle (SDLC). It improved my programming skills, database handling, problemsolving abilities, API development knowledge, and understanding of frontendbackend integration.

Overall, the internship at Fantasy Solution provided an excellent opportunity to apply theoretical concepts in a practical software development environment. The

experience helped me understand industry-oriented development practices and strengthened my technical skills in Java, Spring Boot, MySQL, HTML, CSS, JavaScript and REST API development.

Company Details:

- **Company Name:** Fantasy Solution
- **Industry:** Information Technology (IT) Services and Software Development
- **Location:** # 16,Samanth Plazza,Madurai Main Road,Melapudur,Trichy-1,Tamil Nadu, India
- **Development Technologies:** Java, Spring Boot, HTML, CSS, JavaScript, Bootstrap, MySQL, REST APIs
- **Major Services:** Software Development, Web Application Development, Database Solutions, Mobile Application Development, Software Testing, and Technical Training
- **Internship Training:** Java Full Stack Development, Web Application Development, Database Management, REST API Development, Software Testing, and Real-Time Projects
- **Development Approach:** User-friendly, scalable, secure, reliable, and industry-oriented software solutions
- **Project Completed During Internship**: Car Parking Management System (Java Full Stack)

Services Offered

- Software Development
- Web Application Development
- Website Design and Development
- Mobile Application Development
- Java Full Stack Development
- Database Design and Management
- REST API Development
- Digital Marketing and SEO
- Domain Registration and Web Hosting
- Technical Internship and Industrial Training
- Software Maintenance and Technical Support

Technologies Used by the Company

- **Programming Languages:** Java, JavaScript
- **Frontend:** HTML5, CSS3, JavaScript
- **Backend:** Java Spring Boot
- **Database:** MySQL
- **Development Tools:** Eclipse, IntelliJ IDEA, VS Code
- **Build Tool:** Apache Maven
- **Version Control:** Git and GitHub
- **API Technology:** RESTful Web Services

3. DAILY WORK REPORT

Day 1: Attended orientation session; briefed on internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic – "Travel Agency Management System" – with the faculty in-charge.

Day 2: Conducted requirement gathering by discussing with the mentor. Listed the stakeholders (Customer, Administrator) and identified the core functional requirements of the travel agency application.

Day 3: Carried out a literature review of existing travel booking platforms. Studied their features and documented the drawbacks of manual, paper-based travel agency operations.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements needed for development.

Day 5: Set up the development environment – installed JDK 17, IntelliJ IDEA, Apache Maven, and Git. Verified installation with a sample "Hello World" Spring Boot application.

Day 6: Studied Spring Boot fundamentals – auto-configuration, embedded Tomcat server, annotations such as @RestController, @Service, @Repository, and the standard MVC project structure.

Day 7: Designed the overall system architecture of the application using a layered approach (Presentation Layer, Business Logic Layer, and Data Layer).
Prepared the architecture diagram.

Day 8: Designed the database schema by identifying entities – User, Travel Package, Vehicle, Booking, Payment, and Support Request – along with their attributes.

Day 9: Prepared the Entity-Relationship (ER) diagram showing relationships between User, Booking, Package, Vehicle, and Payment entities.

Day 10: Created wireframes for the Home, Login, and Registration pages.
Planned the responsive layout using the Bootstrap grid system.

Day 11: Developed the Home Page using HTML, CSS, and Bootstrap, including the navigation bar, hero banner, popular package cards, and footer section.

Day 12: Developed the Login Page UI with an email/password form, “Remember Me” option, and client-side validation using JavaScript.

Day 13: Developed the Registration Page UI with fields for name, email, phone, address, role selection, and password confirmation, along with input validations.

Day 14: Initialized the Spring Boot backend project using Spring Initializr.
Organized the project into controller, service, repository, and model packages.

Day 15: Created the Model (entity) classes – User, TravelPackage, and Vehicle – defining their fields and getters/setters.

Day 16: Implemented the UserController and UserService to handle new user registration, including basic input validation and duplicate-email checks.

Day 17: Implemented the AuthController for the login module, including logic in UserService to authenticate users by email and password.

Day 18: Tested the Registration and Login REST APIs using Postman. Verified success and error responses (e.g., invalid credentials, duplicate user).

Day 19: Developed the Package Module – TravelPackage model, PackageController, and PackageService – to support adding, updating, and listing travel packages.

Day 20: Developed the Vehicle Module – Vehicle model, VehicleController, and VehicleService – to manage vehicle type, seating capacity, availability, and rental cost.

Day 21: Built the Package listing and Vehicle listing pages on the frontend and connected them to the respective backend APIs using JavaScript fetch calls.

Day 22: Developed the Booking Module – Booking model, BookingController, and BookingService – to create and store booking records for packages and vehicles.

Day 23: Designed and integrated the Booking Page on the frontend, allowing customers to select a package/vehicle, enter travel details, and submit a booking request.

Day 24: Developed the Payment Module – Payment model, PaymentController, and PaymentService – to process payments and update the transaction status.

Day 25: Designed the Payment Page frontend showing the booking summary and payment method selection, and integrated it with the Payment REST API.

Day 26: Developed the Refund Module, enabling customers to submit refund requests for cancelled bookings and allowing status tracking of each request.

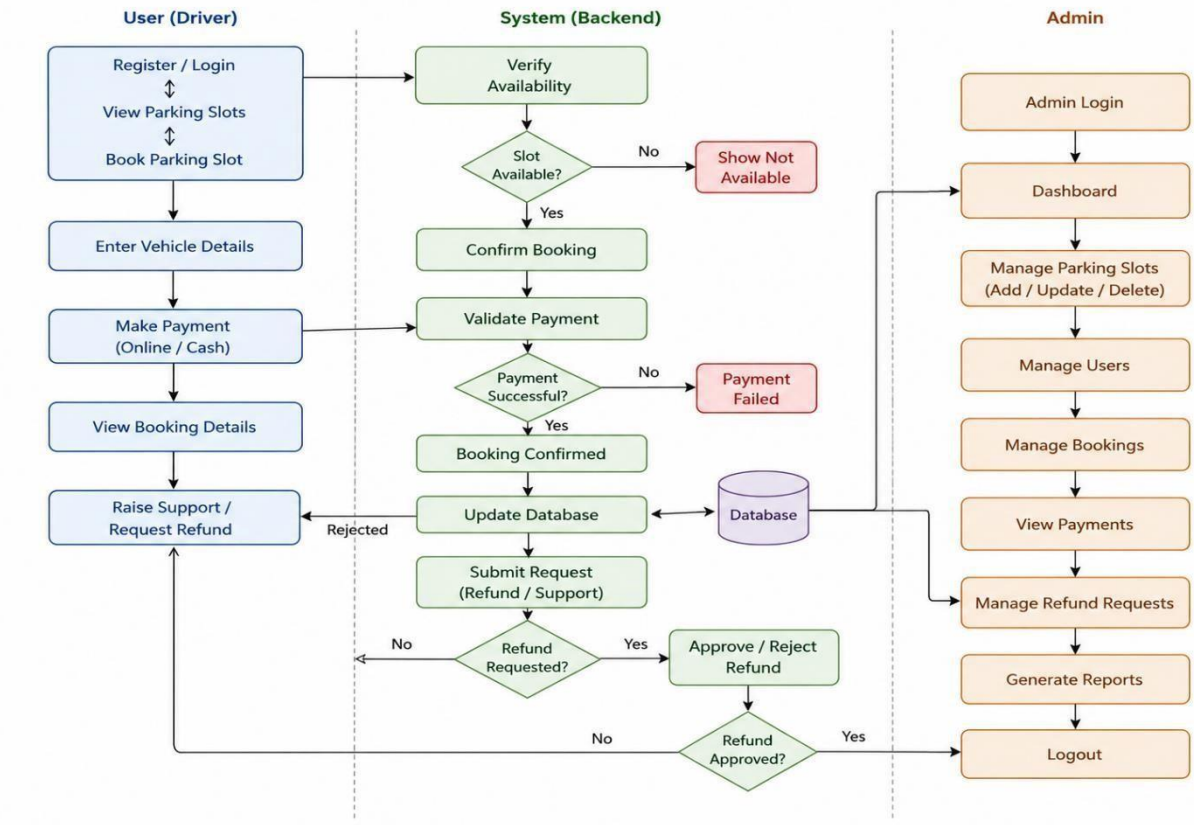
Day 27: Developed the Support Module for raising and managing customer support tickets, enabling communication between customers and administrators.

Day 28: Developed the Notification Module to alert customers about booking confirmation, payment status, cancellations, and support responses.

Day 29: Developed the Admin Module – admin dashboard with statistics, and functionality to manage users, packages, vehicles, bookings, and generate reports.

Day 30: Performed unit and integration testing of all modules, fixed identified bugs, took screenshots for documentation, and compiled the final internship report.

4.BLOCK DIAGRAM



5. CAR PARKING SYSTEM

5.1 Existing System

Most traditional parking facilities still operate through manual and semi-manual processes for managing parking slots, vehicle entries, bookings, and customer

records. Users typically need to visit the parking facility directly or depend on parking staff to check slot availability. Parking details, vehicle information, entry and exit records, and payment details are often maintained using paper registers or separate spreadsheets, with limited automation and centralized management.
This existing approach suffers from several limitations:

- **Manual parking process** – Parking slots are allocated manually by staff, increasing the chances of incorrect slot allocation, booking conflicts, and record errors.

- **No real-time slot availability** – Users cannot instantly check available parking spaces before reaching the parking facility, resulting in unnecessary waiting and difficulty finding suitable slots.

- **Fragmented record-keeping** – Vehicle details, customer information, parking records, and payment information may be maintained in separate registers or files, making data retrieval difficult and time-consuming.

- **Limited accessibility** – Users generally need to visit the parking facility or contact parking staff to enquire about available slots and parking services.

- **Inefficient administrative management** – Administrators may find it difficult to monitor parking occupancy, manage bookings, track vehicles, and generate accurate reports using manual records.

- **Increased waiting time** – Manual slot allocation and verification can create queues during busy periods, reducing the efficiency of the parking facility.

- **Difficulty in tracking parking records –** Maintaining accurate entry, exit, booking, and payment records manually can lead to missing or incorrect information.

- **Limited monitoring and control** – Without a centralized system, administrators have limited visibility into currently occupied, reserved, and available parking slots.

These drawbacks make the existing manual parking system less efficient for handling increasing numbers of vehicles and users. Therefore, there is a need for a digital, centralized, and user-friendly Car Parking Management System that can automate parking operations, improve slot utilization, reduce manual errors, and provide convenient parking services to users.

5.2 Proposed System

The proposed **Car Parking Management System** is a full-stack web application that digitizes and centralizes the major operations of a parking facility. It replaces traditional manual parking procedures with an integrated online platform that can be accessed by users and administrators. The system is designed to simplify parking slot management, reduce waiting time, minimize manual errors, and improve the overall efficiency of parking operations.

The system is developed using **Java Spring Boot** for the backend and **HTML, CSS, JavaScript, and Bootstrap** for the frontend, with **MySQL** used for centralized data storage. Key aspects of the proposed system include:

- **Centralized platform** – User details, vehicle information, parking slots, bookings, and payment records are stored and managed in a single centralized database.

- **User registration and login** – Users can securely register, log in, and access parking services through their personal accounts.
- **Parking slot availability** – Users can view available, occupied, and reserved parking slots, helping them select a suitable parking space before making a booking.
- **Online parking slot booking** – Users can select and reserve available parking slots conveniently through the web application, reducing the need for manual slot allocation.
- **Vehicle management** – Users can add and manage their vehicle details, allowing the system to maintain accurate records for parking operations.
- **Payment management** – The system can maintain parking payment details and provide users with booking and payment information in an organized manner.
- **Administrative dashboard** – Administrators can manage users, parking slots, vehicle details, bookings, payments, and other parking-related activities through a centralized interface.
- **Real-time database updates** – Parking slot status and booking information are updated in the database, helping administrators monitor parking occupancy efficiently.
- **Report and record management** – The system can maintain organized parking records and assist administrators in monitoring bookings, occupied slots, available spaces, and payment information.

By automating these processes, the proposed **Car Parking Management System** reduces manual workload, minimizes booking and slot allocation errors, improves parking space utilization, and provides a convenient and efficient parking experience for both users and administrat

5.3 Capabilities

The Car Parking Management System offers the following key capabilities.

- User Management – Provides secure user registration, login, authentication, and profile management for users and administrators**.**
- Vehicle Management – Enables users to register and manage vehicle details such as vehicle number, type, and other relevant information.
- Parking Slot Availability – Allows users to view available, occupied, and reserved parking slots before making a booking.
- Online Parking Booking – Enables users to select and reserve available parking slots through a simple and convenient booking process.
- Payment Management – Maintains parking payment details and transaction records associated with parking bookings.
- Booking Management – Allows users to view booking details and enables administrators to monitor, update, and manage parking reservations.
- Admin Management – Provides administrators with a centralized dashboard to manage users, parking slots, vehicles, bookings, and payment information.
- Database Management – Stores user, vehicle, parking slot, booking, and payment information securely in a centralized MySQL database.
- Reporting and Monitoring – Helps administrators monitor parking occupancy, booking records, available slots, and other parking-related activitie.

6. PROPOSED METHOD

6.1 User Authentication

- Handles secure registration and login for users and administrators.
- Captures user details such as name, email, phone number, password, and role during registration.
- Validates login credentials and prevents duplicate email registrations.
- Provides role-based access, allowing users to access the booking features and administrators to access the admin dashboard.
- Displays appropriate error messages for invalid login credentials and registration details.

6.2 Parking Slot Management

- Manages parking slot details such as slot number, location, type, and availability status.

- Admin can add, update, and delete parking slots according to parking facility requirements.
- Users can view available, occupied, and reserved parking slots.
- Updates parking slot availability after every successful booking or cancellation.
- Helps prevent double booking by verifying slot availability before confirming a reservation.

6.3 Vehicle Management

- Manages vehicle information such as vehicle number, vehicle type, model, and owner details.
- Users can add, update, and manage their vehicle information.
- Administrators can view and manage registered vehicle details.
- Maintains accurate vehicle records associated with parking bookings.
- Helps identify vehicles and maintain organized parking records.

6.4 Parking Booking Management

- Allows users to select and reserve an available parking slot through the web application.
- Verifies parking slot availability before processing the booking request.
- Records booking details such as user, vehicle, parking slot, booking date, entry time, and exit time.
- Updates the parking slot status automatically after successful booking.
- Allows users to view their booking history and current parking status.
- Helps administrators monitor and manage all parking reservations.

6.5 Payment Management

- Handles payment details associated with parking bookings.
- Records transaction information such as payment ID, amount, payment method, status, and date.

- Updates the booking status after successful payment confirmation.
- Maintains payment records securely in the database.
- Enables administrators to view and monitor parking-related payment transactions.

6.6 Admin Management

- Provides a centralized dashboard for managing the complete parking system.
- Allows administrators to manage users, vehicles, parking slots, bookings, and payment records.
- Enables administrators to monitor available, occupied, and reserved parking spaces.
- Allows administrators to update or remove parking slots and manage booking information.
- Generates useful reports related to parking bookings, slot utilization, users, and payments.

6.7 Database Management

- Stores user, vehicle, parking slot, booking, and payment information in a centralized MySQL database.
- Maintains relationships between users, vehicles, parking slots, and bookings.
- Provides efficient data retrieval and updating through the Spring Boot backend.
- Helps maintain data consistency, accuracy, and security throughout the application.

6.8 Notification and Status Management

- Provides booking status updates to users after successful reservation or cancellation.
- Displays parking slot availability and booking status clearly.
- Provides appropriate messages for successful, failed, or unavailable operations.
- Improves communication between users and administrators and enhances the overall user experience.

7. ALGORITHM

7.1 User Login – Algorithm

1. User enters email and password in the login form.
2. System validates whether all required fields are entered.
3. Login request is sent to the authentication controller.
4. System checks whether the entered email exists in the database.
5. System verifies the entered password with the stored password.
6. If credentials are invalid, an error message is displayed.
7. If credentials are valid, the user is redirected to the appropriate dashboard.

7.2 User Registration – Algorithm

1. User enters name, email, phone number, address, and password.
2. System validates all required registration fields.
3. System checks whether the entered email is already registered.
4. If the email already exists, registration is rejected.

5. If the email is unique, a new user record is created.
6. User details are stored securely in the MySQL database.
7. On successful registration, the user is redirected to the login page.

7.3 Parking Slot Booking – Algorithm

1. User logs into the system and opens the parking slot section.
2. System displays the available parking slots.
3. User selects the required parking slot and enters vehicle details.
4. System verifies the availability of the selected slot.
5. If the slot is unavailable, the user is asked to select another slot.
6. If available, the system creates a new parking booking.
7. The selected slot status is updated as reserved and the booking details are stored.

7.4 Vehicle Management – Algorithm

1. User opens the vehicle management section.
2. User enters vehicle number, vehicle type, model, and other required details.

3. System validates the entered vehicle information.
4. System checks whether the vehicle is already registered.
5. If the vehicle already exists, an appropriate error message is displayed.
6. If the vehicle is new, the details are saved in the database.
7. The updated vehicle information is displayed to the user.

7.5 Payment Processing – Algorithm

1. User confirms the parking booking and proceeds to payment.
2. System calculates and displays the applicable parking fee.
3. User selects a payment method and submits the payment. 4. System verifies the payment amount and booking details.
5. If payment fails, the transaction is marked as FAILED.
6. If payment succeeds, the transaction is marked as SUCCESS and the booking is confirmed.
7. Payment details are stored in the database and confirmation is displayed to the user.

7.6 Booking Management – Algorithm

1. User creates a parking booking by selecting an available slot.

2. System generates a unique booking record.
3. Booking details are stored in the database.
4. System associates the booking with the selected user and vehicle.
5. The parking slot status is updated according to the booking.
6. User can view the current and previous booking details.
7. Administrator can monitor, update, or manage all parking bookings.

7.7 Admin Operations – Algorithm

1. Administrator enters valid login credentials.
2. System verifies the administrator details and opens the admin dashboard.
3. Administrator selects the required management operation.
4. System provides options to manage users, vehicles, parking slots, bookings, and payments.
5. Administrator performs add, update, delete, or view operations.
6. The system validates the changes and updates the MySQL database.
7. The admin dashboard displays the updated system information.

8.CODE IMPLEMENTATION

8.1 Login Module Code

Contains authentication logic for validating user credentials.

```
@RestController

@RequestMapping("/api/auth")

@CrossOrigin(origins = "http://localhost:3000")

public class AuthController {



    @Autowired

    private UserService userService;



    @PostMapping("/login")

    public ResponseEntity<?> login(

            @RequestBody LoginRequest request) {



        User user = userService.authenticateUser(
```

```
            request.getEmail(),
            request.getPassword());

        if (user != null) {
            return ResponseEntity.ok(
                new MessageResponse("Login Successful"));
        } else {
            return ResponseEntity
                .status(HttpStatus.UNAUTHORIZED)
                .body(new MessageResponse(
                    "Invalid Email or Password"));
        }
    }
}
```

8.2 Registration Module Code

Handles new user account creation and validation.

```
@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "http://localhost:3000")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping("/register")
    public ResponseEntity<User> registerUser(
            @RequestBody User user) {

        User newUser = userService.registerUser(user);

        return new ResponseEntity<>(
                newUser,
```

```
            HttpStatus.CREATED);
    }
}
```

8.3 Parking Slot Booking Module Code

Manages parking slot booking and retrieves all available/registered parking bookings.

```
@RestController
@RequestMapping("/api/bookings")
public class BookingController {
    @Autowired
    private BookingService bookingService;     @PostMapping("/create")
    public ResponseEntity<Booking> createBooking(
RequestBody Booking booking) {         Booking
savedBooking =
            bookingService.saveBooking(booking);
        return new ResponseEntity<>(
            savedBooking,
```

```
            HttpStatus.CREATED);
    }
    @GetMapping("/all")
    public List<Booking> getAllBookings() {
        return bookingService.getBookings();
    }
}
```

8.4 Parking Slot Management Module Code

Manages parking slot availability and updates the slot status when a vehicle is parked.

```
@Service
public class ParkingService {

    @Autowired
    private ParkingBookingRepository bookingRepository;

    @Autowired
    private ParkingSlotRepository slotRepository;
```

```
public ParkingBooking bookParking(
        ParkingBooking booking) {

    ParkingSlot slot =
            slotRepository.findById(
                    booking.getSlotId())
            .orElseThrow(() ->
                    new RuntimeException(
                            "Parking Slot Not Found"));

    if (slot.getStatus().equals("OCCUPIED")) {
        throw new RuntimeException(
                "Parking Slot Already Occupied");
    }

    slot.setStatus("OCCUPIED");
    slotRepository.save(slot);

    booking.setBookingStatus("CONFIRMED");

    return bookingRepository.save(booking);
}
```

```
    public List<ParkingBooking> getAllBookings() {
        return bookingRepository.findAll();
    }
}
```

8.5 Payment Module Code

Processes parking fee payments and updates the payment transaction status.

```
@RestController
@RequestMapping("/api/payment")
@CrossOrigin(origins = "http://localhost:3000")
public class PaymentController {

    @Autowired
    private PaymentService paymentService;

    @PostMapping("/process")
    public ResponseEntity<Payment> processPayment(
            @RequestBody Payment payment) {

        Payment savedPayment =
                paymentService.makePayment(payment);

        return new ResponseEntity<>(
                savedPayment,
```

```
            HttpStatus.OK);
    }
}
```

PaymentService.java

```
@Service
public class PaymentService {

    @Autowired
    private PaymentRepository paymentRepository;

    public Payment makePayment(Payment payment) {

        payment.setStatus("SUCCESS");
        payment.setPaymentDate(LocalDate.now());

        return paymentRepository.save(payment);
    }
}
```

8.6 Vehicle Entry and Exit Module Code

Records vehicle entry and exit details and updates the parking slot after the vehicle leaves.

```
@RestController
@RequestMapping("/api/vehicle")
```

```
@CrossOrigin(origins = "http://localhost:3000")
public class VehicleController {

    @Autowired
    private VehicleService vehicleService;

    @PostMapping("/entry")
    public ResponseEntity<VehicleEntry> vehicleEntry(
            @RequestBody VehicleEntry entry) {

        VehicleEntry savedEntry =
                vehicleService.recordEntry(entry);

        return new ResponseEntity<>(
                savedEntry,
                HttpStatus.CREATED);
    }

    @PutMapping("/exit/{id}")
    public ResponseEntity<VehicleEntry> vehicleExit(
            @PathVariable Long id) {

        VehicleEntry exitEntry =
```

```
            vehicleService.recordExit(id);

        return new ResponseEntity<>(
            exitEntry,
            HttpStatus.OK);
    }
}
```

VehicleService.java

```
@Service
public class VehicleService {

    @Autowired
    private VehicleEntryRepository vehicleRepository;

    @Autowired
    private ParkingSlotRepository slotRepository;

    public VehicleEntry recordEntry(
        VehicleEntry entry) {

        entry.setEntryTime(LocalDateTime.now());
        entry.setStatus("PARKED");
```

```
    return vehicleRepository.save(entry);
}

public VehicleEntry recordExit(Long id) {

    VehicleEntry vehicle =
            vehicleRepository.findById(id)
            .orElseThrow(() ->
                    new RuntimeException(
                            "Vehicle Record Not Found"));

    vehicle.setExitTime(LocalDateTime.now());
    vehicle.setStatus("EXITED");

    ParkingSlot slot =
            slotRepository.findById(
                    vehicle.getSlotId())
            .orElseThrow(() ->
                    new RuntimeException(
                            "Parking Slot Not Found"));

    slot.setStatus("AVAILABLE");
    slotRepository.save(slot);
```

```
        return vehicleRepository.save(vehicle);
    }
}
```

8.7 Admin Module Code

Provides administrative operations such as managing parking slots, users, and parking bookings.

```
@RestController
@RequestMapping("/api/admin")
@CrossOrigin(origins = "http://localhost:3000")
public class AdminController {

    @Autowired
    private ParkingSlotRepository slotRepository;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private ParkingBookingRepository bookingRepository;

    @GetMapping("/users")
    public List<User> getUsers() {
```

```
    return userRepository.findAll();
}

@GetMapping("/bookings")
public List<ParkingBooking> getBookings() {
    return bookingRepository.findAll();
}

@PostMapping("/slot")
public ParkingSlot addParkingSlot(
        @RequestBody ParkingSlot slot) {

    slot.setStatus("AVAILABLE");

    return slotRepository.save(slot);
}

@DeleteMapping("/slot/{id}")
public String deleteParkingSlot(
        @PathVariable Long id) {

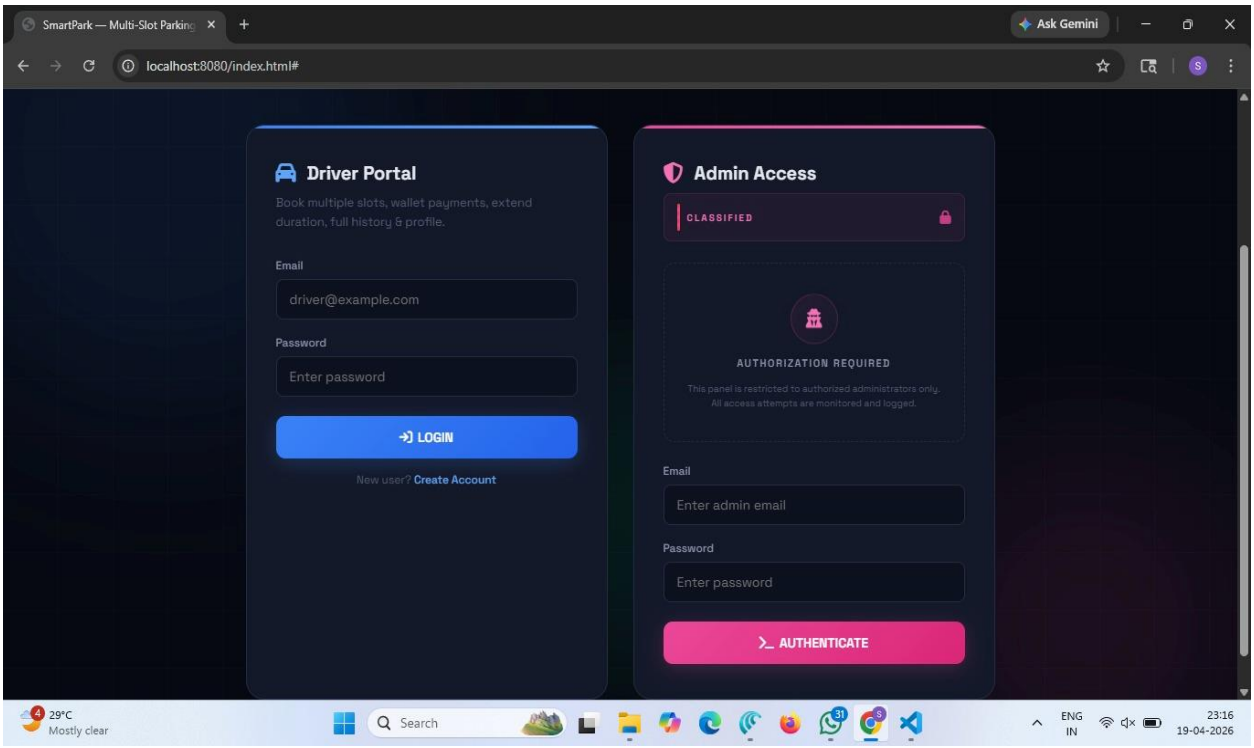
    slotRepository.deleteById(id);
```

```
        return "Parking Slot Deleted Successfully";
    }
}
```

9.SAMPLE OUTPUT

9.1 Home Page

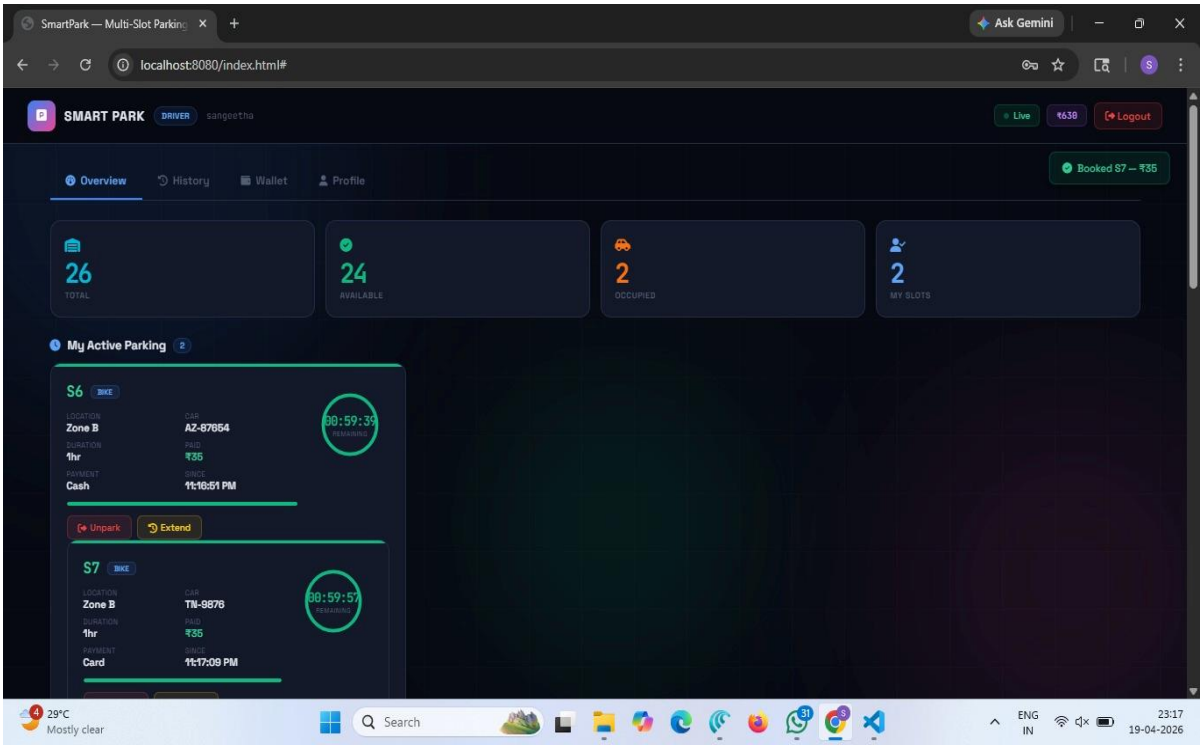
Displays the main interface of the Car Parking Management System.



9. 2 Customer Dashboard

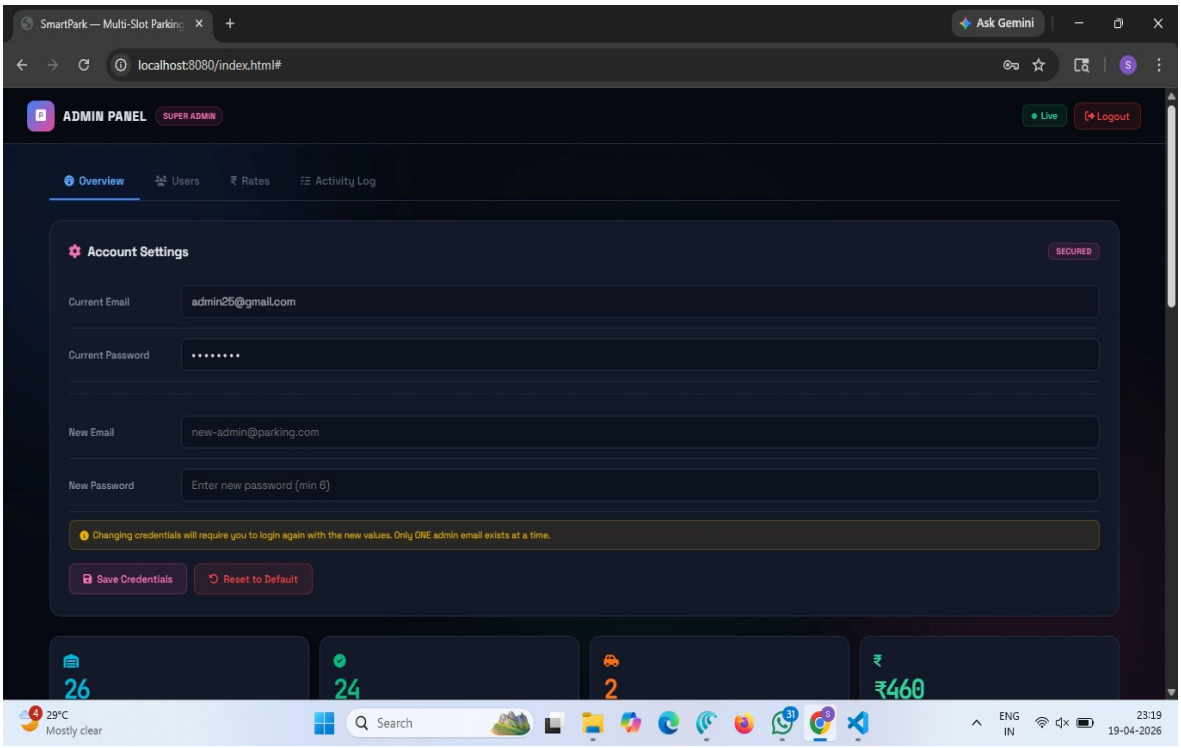
Displays customer activities and parking information. Customers can view available parking slots, book a parking slot, check booking status, view vehicle entry and exit details, make parking fee payments, and manage their profile.

.



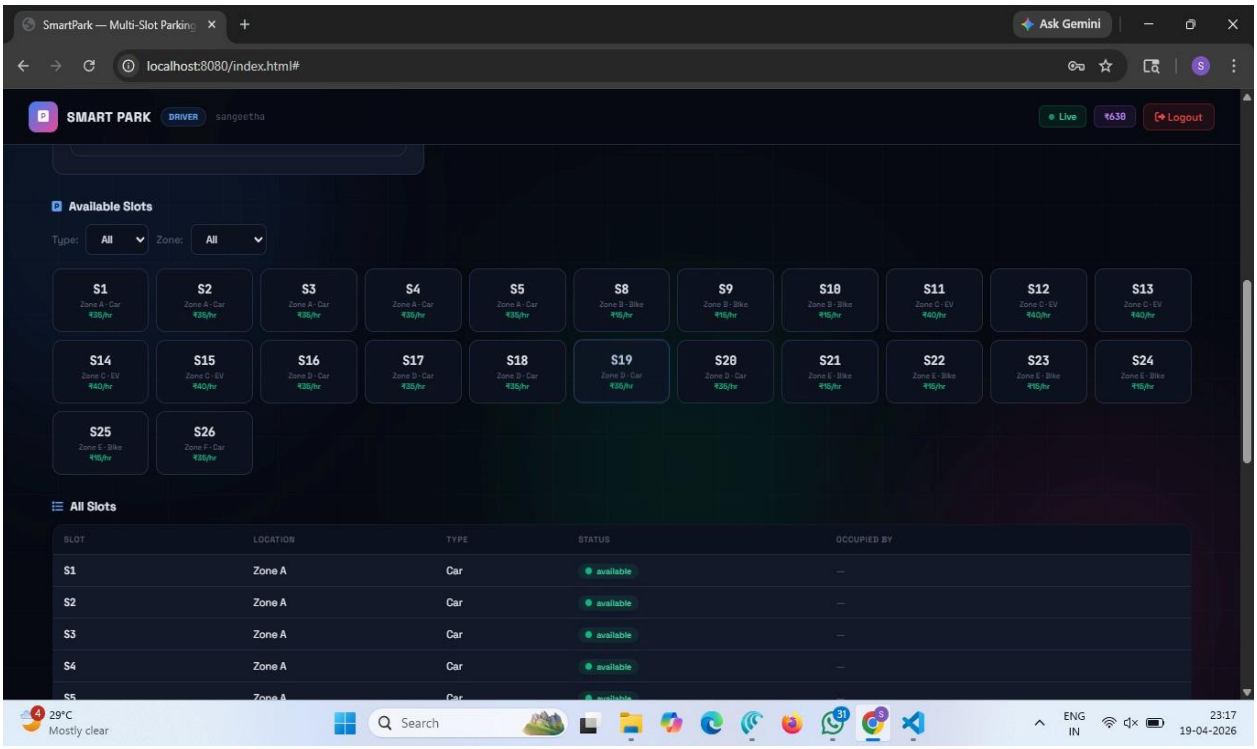
9.3 Admin Dashboard

Provides administrative control over the **Car Parking Management System**. The administrator can manage registered customers, add or remove parking slots, monitor slot availability, view parking bookings, manage vehicle entry and exit records, track payments, and monitor the overall parking system.



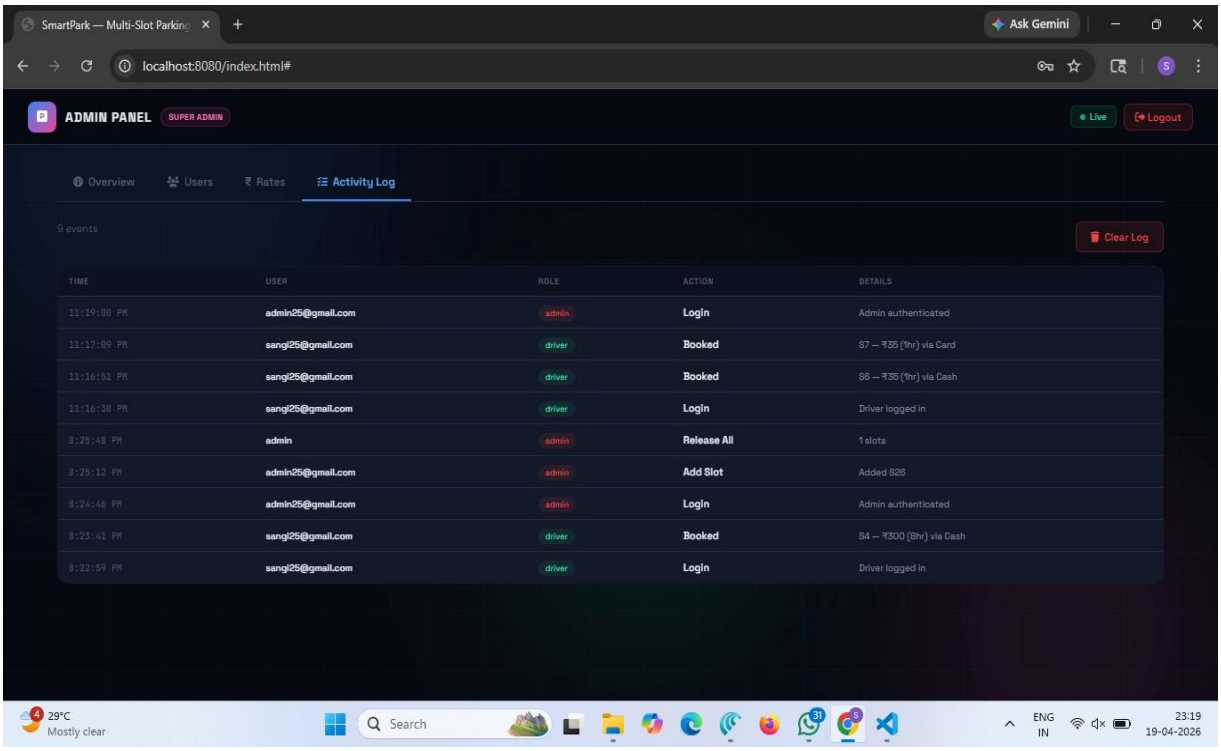
9.4 Parking Slot Management Page

Displays all parking slots and their current availability status. Customers can view available, occupied, and reserved slots before making a booking. The system automatically updates the slot status whenever a vehicle enters or exits the parking area.



9.5 Payment and Booking Status Page

Displays the customer's parking booking details and payment information. Customers can view the booked slot, vehicle number, booking date, parking duration, total parking fee, and payment status. After successful payment, the system updates the booking status and provides confirmation to the customer.



10. ANALYSIS

Testing of the Car Parking Management System was carried out module by module to validate both functional correctness and overall system behavior. Functional analysis confirmed that registration, login, parking slot viewing, slot booking, vehicle entry and exit, payment processing, and admin operations behaved as expected under normal and edge-case inputs such as duplicate emails, invalid credentials, unavailable parking slots, empty fields, and invalid booking details.

Performance observations showed that REST API response times remained fast for typical operations due to the lightweight nature of Spring Boot's embedded server and the current sample data setup. Response times are expected to remain scalable once a full relational database such as MySQL or PostgreSQL is integrated.

Usability analysis indicated that the responsive Bootstrap-based interface adapted well across desktop and mobile screen sizes. The customer dashboard and admin dashboard reduced the number of steps required for parking slot booking, vehicle management, and monitoring compared to traditional manual parking management.

Comparative analysis against the existing manual parking system highlighted clear improvements. Parking slot booking was reduced from a manual process to a few guided clicks, errors in slot allocation were minimized through validation, and administrators gained real-time visibility into parking slots, bookings, vehicles, and payments.

Overall, the analysis confirms that the system meets its objectives of reducing manual effort, improving parking management accuracy, minimizing booking errors, and providing a convenient and efficient platform for both customers and administrators.

11. CONCLUSION

The **Car Parking Management System** successfully demonstrates how full-stack web technologies can be applied to modernize and streamline parking management operations. By replacing manual, paper-based parking processes with a centralized digital platform, the system reduces administrative workload, minimizes human errors, and improves the overall parking experience for customers.

The project's layered architecture, consisting of a responsive Bootstrap-based frontend and a Spring Boot backend, provides a clear separation of concerns and enables smooth communication through REST APIs. Core modules including user registration, authentication, parking slot

management, slot booking, vehicle entry and exit, payment processing, and administrative management were implemented and tested successfully.

The system provides customers with an easy way to check parking slot availability, reserve a slot, manage vehicle details, and complete parking payments. At the same time, administrators can efficiently monitor parking slots, bookings, vehicles, users, and payment information.

This project also provided valuable practical experience in full-stack web development, REST API design, Spring Boot, database interaction, responsive UI development, layered architecture, and software testing. The developed system can serve as a foundation for a more advanced smart parking solution in the future.

12. FUTURE WORK

- Integration with a production-grade relational database such as **MySQL/PostgreSQL** for secure and reliable data storage.
- Integration of a real payment gateway such as **Razorpay or Stripe** for live parking fee transactions.
- Development of a dedicated **Android and iOS mobile application** for convenient parking reservations.

- Implementation of **QR code-based parking entry and exit** for faster vehicle verification.
- Integration of **IoT-based parking sensors** to detect real-time parking slot occupancy.
- Implementation of **GPS and location-based parking search** to help users find nearby parking facilities.
- Automated **email and SMS notifications** for booking confirmation, payment, entry, and exit updates.
- Cloud deployment using **AWS, Azure, or GCP** for improved scalability and availability.
- Development of advanced **analytics and revenue dashboards** for administrators.
- Implementation of **AI-based parking recommendations** based on user location, availability, and parking preferences.
- Integration of **number plate recognition (ANPR)** for automatic vehicle identification.
- Implementation of **JWT-based authentication and role-based access control** for enhanced system security.

13. REFERENCES

1. **W3Schools** – HTML, CSS, JavaScript, and Bootstrap learning resources.
2. **GeeksforGeeks** – Java, Spring Boot, REST API, and database programming concepts.
3. **Spring Boot Official Documentation** – Backend development and REST API implementation.
4. **Oracle Java Documentation** – Java language features and standard libraries.
5. **Bootstrap Documentation** – Responsive UI design and frontend components.
6. **Spring Data JPA Reference Documentation** – Repository layer and database interaction.
7. **MySQL Documentation** – Relational database management and SQL concepts.
8. **MDN Web Docs** – HTML, CSS, JavaScript, and web development

Fantasy Solution

Find ur Passion...

Date: 28.05.2026

Keerthana R,
Reg.No: 810023205068,
Department of Information Technology,
University College Of Engineering,
Bharathidasan Institute of Technology (BIT) Campus,
Tiruchirappalli - 620 024.

Dear Keerthana,

Sub: Internship

On behalf of **FANTASY SOLUTION**. We would like to inform you that you have been accepted as one of our interns. We are pleased to inform you that you have been accepted as per the company requirements for the internship with **FULL STACK DEVELOPMENT**.

Your internship wil l begin from **01st June 2026 to 30th June 2026**. You would assign to various tasks which relates to the project assigned to you after whom your performance would be assessed and appraised.

FANTASY SOLUTION TRICHY-1

Authorized Signature

(With Seal)

#16, Samnath Plazza, 3rd Floor, Melapudur, Trichy - 620 001.
Ph : 0431-4250535 | Mob : 90430 95535
www.fantasysolution.in | info@fantasysolution.in

FANTASY SOLUTION

Fins Us Passion...

SOFTWARE DEVELOPMENT AND TRAINING

Certificate of Accomplishment

This is certify that Mr./Miss. KEERTHANA R (8100232050 68)

Student of UNIVERSITY COLLEGE OF ENGINEERING

BIT CAMPUS ANNA UNIVERSITY - TRICHY

underwent Inplant Training / Internship / Course on FULL STACK DEVELOPMENT

concern from 01/06/2026 to 30/06/2026 issued on 30/06/2026.

Technical Incharge

Manager

CEO

FANTASY SOLUTION TRICHY-1

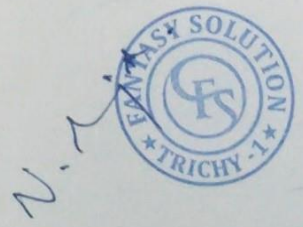
#16, Samnath Plazza, 3rd Floor, Madurai Main Road, Melapudur, Trichy-1.
0431 358 3860 fantazysolution@gmail.com www.fantasysolution.in

ATTENDANCE CERTIFICATE FOR INDUSTRIAL TRAINING / INTERNSHIP

(Attendance Certificate to be signed by the competent authority of the industry mentioning the period of Industrial Training / Internship)

The Student Mr./ Ms. **KEERTHANA R** (Reg.No.) **810023205068**,

Studying **FULL STACK DEVELOPMENT** programme at **UNIVERSITY COLLEGE OF ENGINEERING BIT CAMPUS ANNA UNIVERSITY, TIRUCHIRAPPALLI** College in semester **7th** at Department of **INFORMATION TECHNOLOGY** has attended Industrial Training / Internship from **01/06/2026** to **30/06/2026**. It is certified that he / she has completed / not completed the Industrial Training / Internship in our **FANTASY SOLUTION** organization / institution.



Signature

Name and Designation of the Officer
Seal of the Organization